COMP102: DATA STRUCTURES WITH PYTHON

Complements to: Analysis of Algorithms

Rachad Bentbib

March 25th, 2026

Plan

1 Motivation

Data Structures
Algorithms
Space Usage
References

2 Analysis of Algorithms

The RAM Model
Asymptotic Analysis
Time and Space Costs
Correctness

Motivation

Data Structures

Motivation: Why Data Structures Matter

Problem: Find the Unique Letters in a String

Input:

- A string s of length n

Output:

- The distinct characters appearing in s

Example:

$$\texttt{"banana"} \longrightarrow \{\texttt{b},\texttt{a},\texttt{n}\}$$

Question:

Does the choice of data structure affect performance?

Two Solutions: List vs Set

Python illustration

Using a List

```python
def unique_letters_list(s):
    result = []
    for ch in s:
        if ch not in result:
            result.append(ch)
    return result
```

Using a Set

```python
def unique_letters_set(s):
    result = set()
    for ch in s:
        result.add(ch)
    return result
```

Two Solutions: List vs Set

Python illustration

Using a List

```python
def unique_letters_list(s):
    result = []
    for ch in s:
        if ch not in result:
            result.append(ch)
    return result
```

Using a Set

```python
def unique_letters_set(s):
    result = set()
    for ch in s:
        result.add(ch)
    return result
```

Cost of key operation

- List membership check: $\Theta(n)$
- Set insertion: $O(1)$

Two Solutions: List vs Set

Python illustration

Using a List

```python
def unique_letters_list(s):
    result = []
    for ch in s:
        if ch not in result:
            result.append(ch)
    return result
```

Using a Set

```python
def unique_letters_set(s):
    result = set()
    for ch in s:
        result.add(ch)
    return result
```

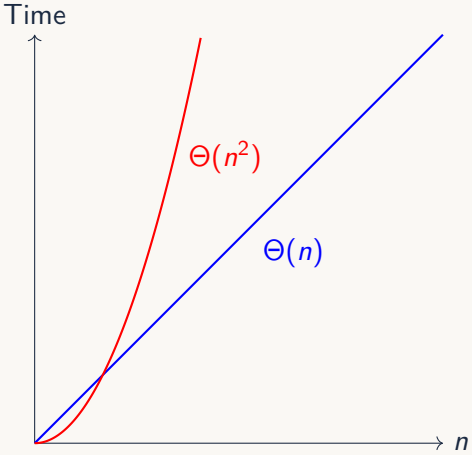
Cost of key operation

- List membership check: $\Theta(n)$
- Set insertion: $O(1)$

Total running time

List version: $\Theta(n^2)$ Set version: $\Theta(n)$

Time Growth: List vs Set



For the same problem, the choice of the data structure impacts performance dramatically!

Algorithms

Motivation: Why Algorithms Matter

Problem: Search in a Sorted List

Input:

- Sorted list A of size n
- Target value x

Output:

- Index of x or NIL

Question:

Does the search strategy affect performance?

Two Algorithms: Linear vs Binary Search

Python illustration

Linear Search

```python
def linear_search(A, x):
    for i in range(len(A)):
        if A[i] == x:
            return i
    return None
```

Binary Search

```python
def binary_search(A, x):
    left, right = 0, len(A)-1
    while left <= right:
        mid = (left+right)//2
        if A[mid] == x:
            return mid
        elif A[mid] < x:
            left = mid + 1
        else:
            right = mid - 1
    return None
```

Two Algorithms: Linear vs Binary Search

Python illustration

Linear Search

```python
def linear_search(A, x):
    for i in range(len(A)):
        if A[i] == x:
            return i
    return None
```

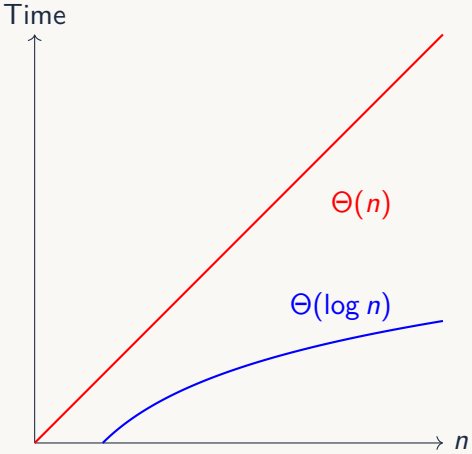
Binary Search

```python
def binary_search(A, x):
    left, right = 0, len(A)-1
    while left <= right:
        mid = (left+right)//2
        if A[mid] == x:
            return mid
        elif A[mid] < x:
            left = mid + 1
        else:
            right = mid - 1
    return None
```

Total running time

Linear Search $O(n)$ Binary Search $O(\log n)$

Time Growth: Linear vs Binary Search



For the same problem, the choice of the algorithm impacts performance dramatically!

Space Usage

Motivation: Space Complexity

Problem: Computing a certain Fibonacci number

The Fibonacci sequence is defined recursively by

$$F_0 = 1, \qquad F_1 = 1$$

$$F_n = F_{n-1} + F_{n-2} \qquad \text{for } n \geq 2$$

The first terms of the sequence are: $1,\ 1,\ 2,\ 3,\ 5,\ 8,\ 13, \ldots$

Problem

- **Input:** an integer $n \geq 0$
- **Output:** the n-th Fibonacci number F_n

How can we compute F_n with minimal storage space?

Fibonacci: Two Algorithms

Python illustration

Arrays

```python
def fib_array(n):
    F = [1,1]
    for i in range(2,n+1):
        F.append(F[i-1] + F[i-2])
    return F[n]
```

Constant Memory

```python
def fib_constant(n):
    a,b = 0,1
    for _ in range(n):
        a,b = b,a+b
    return a
```

Fibonacci: Two Algorithms

Python illustration

Arrays

```python
def fib_array(n):
    F = [1,1]
    for i in range(2,n+1):
        F.append(F[i-1] + F[i-2])
    return F[n]
```

Constant Memory

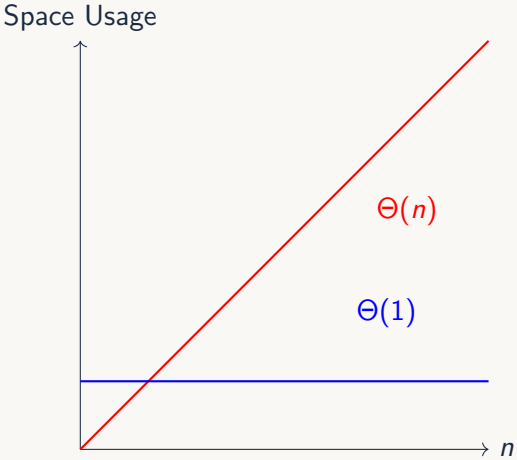
```python
def fib_constant(n):
    a,b = 0,1
    for _ in range(n):
        a,b = b,a+b
    return a
```

Total space:

$\Theta(n)$

$\Theta(1)$

Space Usage Growth



For the same problem, the algorithm design can reduce space usage considerably!

References

Main References

- **M. T. Goodrich, R. Tamassia, M. H. Goldwasser**
  Data Structures & Algorithms in Python, Wiley, 2013.
- **T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein**
  Introduction to Algorithms, 3rd edition, MIT Press, 2009.
  Available in the library for consultation on site.
  A fourth edition was published in 2022.

Analysis of Algorithms

The RAM Model

Why a Computational Model?

To analyze algorithms mathematically we need:

- a precise notion of **memory**;
- a precise notion of **elementary operations**;
- a precise notion of **cost**.

The RAM model provides a suitable abstraction of real computers.

Definition of the RAM Model

Definition 2.1 Random Access Machine (RAM)

A RAM machine consists of:

- An infinite sequence of memory cells

$$M[0], M[1], M[2], \ldots$$

- Each cell stores a word of fixed size
- A finite set of registers (with finite memory)
- A finite set of elementary instructions

Memory Layout

$M[0]$	$M[1]$	$M[2]$	$M[3]$	$M[4]$	$M[5]$	$M[6]$	$M[7]$	$M[8]$	$M[9]$	$\cdots$
0	1	2	3	4	5	6	7	8	9	

Memory addresses

All memory cells are addressable in constant time.

Constant-Time Assumption

In the RAM model:

- Accessing $M[i]$ costs $O(1)$
- Arithmetic operations cost $O(1)$
- Assignment costs $O(1)$
- Comparison costs $O(1)$

Variables as references

A variable is a reference (memory address) to the data representing an object .

Both x and y refer to the same memory block.

Variables as references

Python illustration

```python
x = 1
y = x
print(id(x) == id(y))
```

Output:

Variables as references

Python illustration

```python
x = 1
y = x
print(id(x) == id(y))
```

Output:

```
True
```

Variables as references

Python illustration

```python
def set_first_zero(l):
    l[0] = 0


x = [1, 2, 3]
set_first_zero(x)
print(x)
```

Output:

Variables as references

Python illustration

```python
def set_first_zero(l):
    l[0] = 0


x = [1, 2, 3]
set_first_zero(x)
print(x)
```

Output:

```
[0, 2, 3]
```

Variables as references

Python illustration

```python
a = 0
def increment(n):
    print(f"Same? {id(a) == id(n)}")
    n = n + 1
    print(f"Same after increment? {id(a) == id(n)}")


increment(a)
print(f"a = {a}")
```

Output:

Variables as references

Python illustration

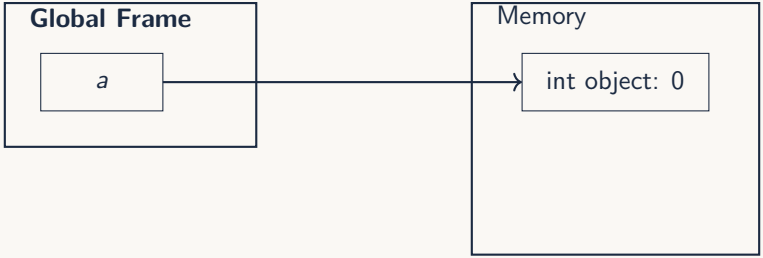
```python
a = 0
def increment(n):
    print(f"Same? {id(a) == id(n)}")
    n = n + 1
    print(f"Same after increment? {id(a) == id(n)}")


increment(a)
print(f"a = {a}")
```

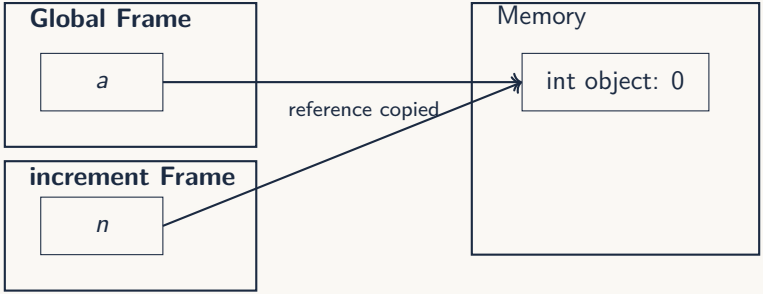
Output:

```
Same? True
Same after increment? False
a = 0
```

Pass-by-Object-Reference in Python

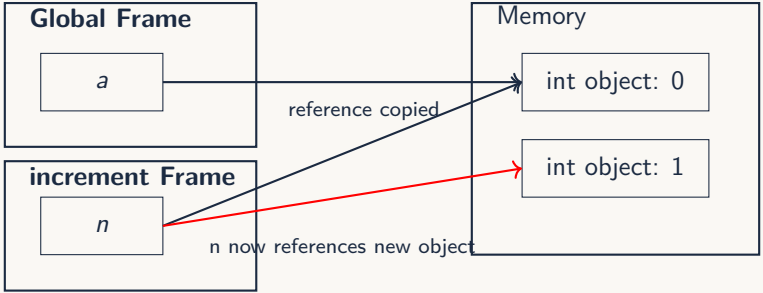


Pass-by-Object-Reference in Python



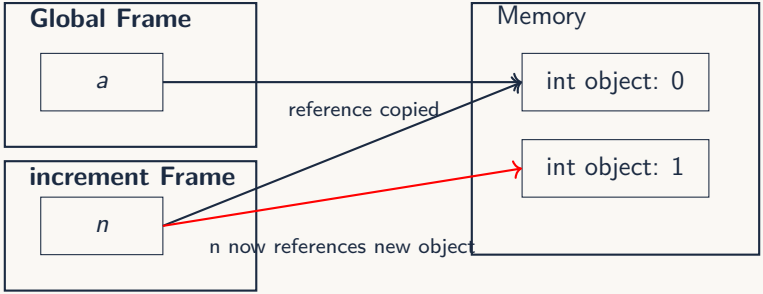
- Initially, both *a* and *n* reference the same integer object (0).

Pass-by-Object-Reference in Python



- Initially, both a and n reference the same integer object (0).
- Integers are **immutable**.
- The statement $n = n + 1$ creates a new object (1) and rebinds n.
- a still references the original object.

Pass-by-Object-Reference in Python



- Initially, both a and n reference the same integer object (0).
- Integers are **immutable**.
- The statement $n = n + 1$ creates a new object (1) and rebinds n.
- a still references the original object.
- The global frame also contains 'increment' as a reference to a function object, not shown for simplicity.

Asymptotic Analysis

Motivation

We want to classify asymptotic growth rates

$$n, \quad n \log n, \quad n^2, \quad 2^n$$

ignoring:

- Constant factors
- Lower-order terms

Big-Oh Notation

Definition 2.2 Big-Oh

Let $g : \mathbb{N} \to \mathbb{R}^+$.

$$O(g) := \{f : \mathbb{N} \to \mathbb{R}^+ \text{ s.t. } \exists c > 0, \exists n_0, \forall n \geq n_0, 0 \leq f(n) \leq cg(n)\}$$

Example 2.1.

$n + 1$ and $2n^2 + n$ are $O(n^2)$.

Big-Oh Notation

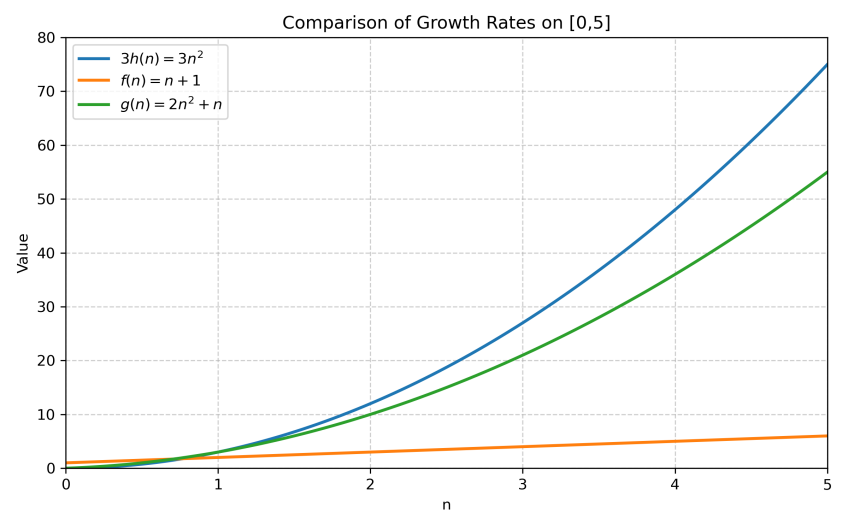


Figure: Some $O(n^2)$ functions

Important Notational Precision

- $f(n) = O(g(n))$ is notation for $f \in O(g)$.
- $f(n) + O(g(n))$ means $f(n) + h(n)$ for some $h \in O(g)$.

This is NOT arithmetics with sets.

Theta Definition

Definition 2.3 Theta

Let $g : \mathbb{N} \to \mathbb{R}^+$.

$$\Theta(g) := \{f : \mathbb{N} \to \mathbb{R}^+ \text{ s.t. } \exists c_1, c_2 > 0, \exists n_0, \forall n \geq n_0, c_1 g(n) \leq f(n) \leq c_2 g(n)\}$$

Example 2.2.

$2n^2 + n$ is $\Theta(n^2)$, while $n + 1$ is not $\Theta(n^2)$.

Theta Notation

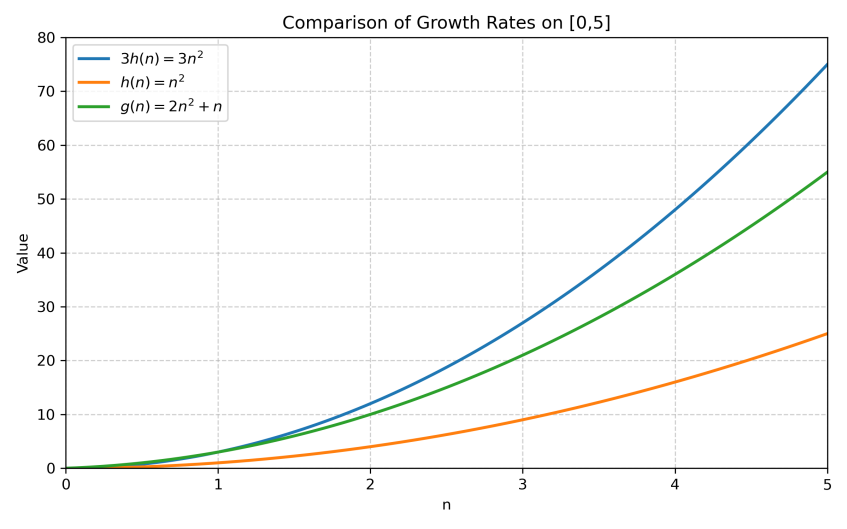


Figure: The function g is $\Theta(n^2)$ function

Little-o

Definition 2.4 Little-o

Let $g : \mathbb{N} \to \mathbb{R}^+$.

$$o(g) := \left\{ f : \mathbb{N} \to \mathbb{R}^+ \text{ s.t. } \lim_{n\to\infty} \frac{f(n)}{g(n)} = 0 \right\}$$

Hierarchy of Growth Rates

Notation: $f(n) \prec g(n)$ means $f = o(g)$.

Proposition 2.1 Asymptotic Growth Hierarchy

For constants $a > 0$ and $b > a$:

$$1 \prec \log\log n \prec (\log n)^b \prec n^a \prec n^a \log n \prec n^b \prec 2^n \prec n^n$$

Hierarchy of Growth Rates

Notation: $f(n) \prec g(n)$ means $f = o(g)$.

Proposition 2.3 Asymptotic Growth Hierarchy

For constants $a > 0$ and $b > a$:

$$1 \prec \log\log n \prec (\log n)^b \prec n^a \prec n^a \log n \prec n^b \prec 2^n \prec n^n$$

Proposition 2.4 Stirling's formula

One has $n! = \sqrt{2\pi n}\left(\frac{n}{e}\right)^n \left(1 + \Theta\left(\frac{1}{n}\right)\right)$.
In particular, $2^n \prec n! \prec n^n$, and $\log(n!) = \Theta(n \log n)$.

Time and Space Costs

Pseudocode Principles

Pseudocode Principles

- Language-independent

Pseudocode Principles

- Language-independent
- Explicit actions

Pseudocode Principles

- Language-independent
- Explicit actions
- Abstract data types

Example: Linear Search

Algorithm 1: Linear Search

Input: Array A of size n, value x
Output: Index of x or NIL

```
i ← 0
while i < n do
    if A[i] = x then
        return i
    i ← i + 1
return NIL
```

Example: Linear Search (Worst Case)

Worst case: x is not in A.

Table: Cost analysis of Linear Search (worst case)

Line	Cost	Times Executed
1: $i \leftarrow 0$	c_1	1
2: $i < n$ (while test)	c_2	$n + 1$
3: $A[i] = x$	c_3	n
5: $i \leftarrow i + 1$	c_5	n
6: return NIL	c_6	1

Example: Linear Search (Worst Case)

Worst case: x is not in A.

Table: Cost analysis of Linear Search (worst case)

Line	Cost	Times Executed
1: $i \leftarrow 0$	c_1	1
2: $i < n$ (while test)	c_2	$n+1$
3: $A[i] = x$	c_3	n
5: $i \leftarrow i+1$	c_5	n
6: return NIL	c_6	1

Total running time:

$$\begin{aligned} T_{\text{worst}}(n) &= c_1 + (n+1)c_2 + nc_3 + nc_4 + c_6 \\ &= (c_2 + c_3 + c_4)n + (c_1 + c_2 + c_6) \\ &= \Theta(n) \end{aligned}$$

Example: Linear Search (Best Case)

Best case: $x = A[0]$.

Table: Cost analysis of Linear Search (best case)

Line	Cost	Times Executed
1: $i \leftarrow 0$	c_1	1
2: $i < n$ (while test)	c_2	1
3: $A[i] = x$	c_3	1
4: return i	c_4	1

Example: Linear Search (Best Case)

Best case: $x = A[0]$.

Table: Cost analysis of Linear Search (best case)

Line	Cost	Times Executed
1: $i \leftarrow 0$	c_1	1
2: $i < n$ (while test)	c_2	1
3: $A[i] = x$	c_3	1
4: return i	c_4	1

Total running time:

$$\begin{aligned} T_{\text{best}}(n) &= c_1 + c_2 + c_3 + c_4 \\ &= O(1) \end{aligned}$$

Example: Linear Search (Average Case)

The average case depends on the probability space: the possible scenarios and their probability distribution.

Example: Linear Search (Average Case)

The average case depends on the probability space: the possible scenarios and their probability distribution.
We determine the average time complexity for the scenarios where x is an element of an array of n elements that can uniformly occupy any index from 0 to $n - 1$.

Example: Linear Search (Average Case)

The average case depends on the probability space: the possible scenarios and their probability distribution.
We determine the average time complexity for the scenarios where x is an element of an array of n elements that can uniformly occupy any index from 0 to $n-1$.
The sample space is therefore $\Omega := \{0, \ldots, n-1\}$, and the probability space is the discrete probability space on Ω with a uniform probability distribution: $(\Omega, \mathcal{P}(\Omega), P)$ where $\mathcal{P}(\Omega)$ is the powerset of Ω, and P is the probability distribution satisfying $P(\{k\}) = \frac{1}{n}$ for every $k \in \Omega$.

Example: Linear Search (Average Case)

Assume $0 \leq k \leq n-1$ and x is at position k.

Table: Cost analysis of Linear Search when x is at position k

Line	Cost	Times Executed
1: $i \leftarrow 0$	c_1	1
2: $i < n$ (while test)	c_2	$k+1$
3: $A[i] = x$	c_3	$k+1$
4: return i	c_4	1
5: $i \leftarrow i+1$	c_5	k

Example: Linear Search (Average Case)

Assume $0 \leq k \leq n-1$ and x is at position k.

Table: Cost analysis of Linear Search when x is at position k

Line	Cost	Times Executed
1: $i \leftarrow 0$	c_1	1
2: $i < n$ (while test)	c_2	$k+1$
3: $A[i] = x$	c_3	$k+1$
4: return i	c_4	1
5: $i \leftarrow i+1$	c_5	k

Total running time:

$$\begin{aligned} T_k(n) &= c_1 + (k+1)c_2 + (k+1)c_3 + c_4 + kc_5 \\ &= \underbrace{(c_2 + c_3 + c_5)}_{a} k + \underbrace{(c_1 + c_2 + c_3 + c_4)}_{b} = ak + b \end{aligned}$$

Example: Linear Search (Average Case)

Below n is seen as a parameter, hence $T_k(n)$ is a function of $k \in \Omega$.

$$
\begin{aligned}
\mathbb{E}[T] &= \sum_{k \in \Omega} P(\{k\}) T_k(n) \\
&= \frac{1}{n} \sum_{k=0}^{n-1} T_k(n) \\
&= \frac{1}{n} \sum_{k=0}^{n-1} (ak + b) \\
&= \frac{1}{n} \left(a \sum_{k=0}^{n-1} k + \sum_{k=0}^{n-1} b \right) \\
&= \frac{1}{n} \left(a \frac{n(n-1)}{2} + nb \right) \\
&= \frac{a}{2}(n-1) + b \\
&= \Theta(n)
\end{aligned}
$$

Space Complexity

Definition 2.5 Space Complexity

The space complexity of an algorithm is the total amount of memory used as a function of the input size n.

Space Complexity

Definition 2.6 Space Complexity

The space complexity of an algorithm is the total amount of memory used as a function of the input size n.

We distinguish:

- **Input space:** memory required to store the input (included depending on convention);

Space Complexity

Definition 2.7 Space Complexity

The space complexity of an algorithm is the total amount of memory used as a function of the input size n.

We distinguish:

- **Input space:** memory required to store the input (included depending on convention);
- **Auxiliary space:** extra memory used by the algorithm.

Example: Linear Search

Recall the algorithm:

- Input: array A of size n, input variable x
- Local variables: i

At every instant, memory usage is:

$$\begin{aligned} \text{Array } A &: \Theta(n) \\ \text{Variable } x &: \Theta(1) \\ \text{Variable } i &: \Theta(1) \end{aligned}$$

Total space: $S(n) = \Theta(n) + \Theta(1) + \Theta(1)$, which is $\Theta(n)$.
Auxiliary space: $O(1)$.

Correctness

Correctness of an Algorithm

Definition 2.8 Algorithm Correctness

An algorithm is **correct** if for every valid input it:

- terminates, and
- produces the correct output.

Loop Invariants

Definition 2.9 Loop Invariant

A **loop invariant** is a property that holds before and after every iteration of a loop.

Intuitively:

- It describes the **partial progress** of the algorithm.
- It captures what **portion of the problem** has already been solved.

How Do We Prove Correctness?

For algorithms containing loops, correctness is usually proved using a loop invariant.

How Do We Prove Correctness?

For algorithms containing loops, correctness is usually proved using a loop invariant.

The proof structure follows three steps:

1. **Initialization**
   The invariant is true before the loop starts.

How Do We Prove Correctness?

For algorithms containing loops, correctness is usually proved using a loop invariant.

The proof structure follows three steps:

1. **Initialization**
   The invariant is true before the loop starts.

2. **Maintenance**
   If the invariant is true before an iteration, it remains true after the iteration.

How Do We Prove Correctness?

For algorithms containing loops, correctness is usually proved using a loop invariant.

The proof structure follows three steps:

1. **Initialization**
   The invariant is true before the loop starts.

2. **Maintenance**
   If the invariant is true before an iteration, it remains true after the iteration.

3. **Termination**
   When the loop stops, the invariant implies the algorithm produces the correct result.

Vacuous Truth

Let U be the universe of elements we quantify over.
Let A be a set of elements in U, and P be a unary predicate.

Vacuous Truth

Let U be the universe of elements we quantify over.
Let A be a set of elements in U, and P be a unary predicate.

- The expression $\forall x \in A, P(x)$ is interpreted as $\forall x \in U, (x \in A \implies P(x))$.

Vacuous Truth

Let U be the universe of elements we quantify over.
Let A be a set of elements in U, and P be a unary predicate.

- The expression $\forall x \in A, P(x)$ is interpreted as $\forall x \in U, (x \in A \implies P(x))$.
  In particular, $\forall x \in \varnothing, P(x)$ is interpreted as $\forall x \in U, (x \in \varnothing \implies P(x))$, which is true. Indeed, for any $x \in U$, the statement $x \in \varnothing$ is false, hence the material conditional $x \in \varnothing \implies P(x)$ is true.

Vacuous Truth

Let U be the universe of elements we quantify over.
Let A be a set of elements in U, and P be a unary predicate.

- The expression $\forall x \in A, P(x)$ is interpreted as $\forall x \in U, (x \in A \implies P(x))$.
  In particular, $\forall x \in \varnothing, P(x)$ is interpreted as $\forall x \in U, (x \in \varnothing \implies P(x))$, which is true. Indeed, for any $x \in U$, the statement $x \in \varnothing$ is false, hence the material conditional $x \in \varnothing \implies P(x)$ is true.
  This is called **vacuous truth**.

Vacuous Truth

Let U be the universe of elements we quantify over.
Let A be a set of elements in U, and P be a unary predicate.

- The expression $\forall x \in A, P(x)$ is interpreted as $\forall x \in U, (x \in A \implies P(x))$.
  In particular, $\forall x \in \varnothing, P(x)$ is interpreted as $\forall x \in U, (x \in \varnothing \implies P(x))$, which is true. Indeed, for any $x \in U$, the statement $x \in \varnothing$ is false, hence the material conditional $x \in \varnothing \implies P(x)$ is true.
  This is called **vacuous truth**.
- On the other hand, the expression $\exists A, P(x)$ is interpreted as $\exists x \in U, (x \in A) \wedge P(x)$.

Vacuous Truth

Let U be the universe of elements we quantify over.
Let A be a set of elements in U, and P be a unary predicate.

- The expression $\forall x \in A, P(x)$ is interpreted as $\forall x \in U, (x \in A \implies P(x))$.
  In particular, $\forall x \in \varnothing, P(x)$ is interpreted as $\forall x \in U, (x \in \varnothing \implies P(x))$, which is true. Indeed, for any $x \in U$, the statement $x \in \varnothing$ is false, hence the material conditional $x \in \varnothing \implies P(x)$ is true.
  This is called **vacuous truth**.
- On the other hand, the expression $\exists A, P(x)$ is interpreted as $\exists x \in U, (x \in A) \wedge P(x)$.
  In particular, any statement of the form $\exists x \in \varnothing, P(x)$ is false because it amounts to $\exists x \in U, (x \in \varnothing) \wedge P(x)$.

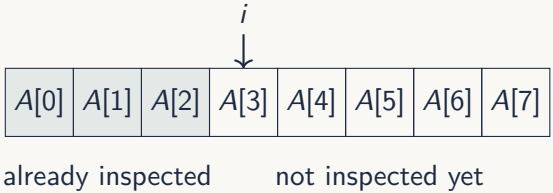
Loop Invariant for Linear Search

Loop invariant:

At the start of iteration i, we have $\forall j \in \{0, \ldots, i-1\}, x \neq A[j]$.

In words: 'The element x does not appear among the elements already inspected.'

Loop Invariant for Linear Search



Loop invariant

$$x \notin \{A[0], A[1], \ldots, A[i-1]\}$$

The shaded region contains elements already checked.

Proof of Correctness

Initialization

Before the first iteration, $i = 0$. No elements have been inspected, thus the invariant holds vacuously: $\forall j \in \varnothing, x \neq A[j]$ is true.

Proof of Correctness

Initialization

Before the first iteration, $i = 0$. No elements have been inspected, thus the invariant holds vacuously: $\forall j \in \varnothing, x \neq A[j]$ is true.

Maintenance

Before iteration i, we have $x \notin \{A[0], \ldots, A[i-1]\}$. If $A[i] \neq x$, the algorithm increments i and we have $x \notin \{A[0], \ldots, A[i]\}$, hence the invariant still holds.

Proof of Correctness

Initialization

Before the first iteration, $i = 0$. No elements have been inspected, thus the invariant holds vacuously: $\forall j \in \varnothing, x \neq A[j]$ is true.

Maintenance

Before iteration i, we have $x \notin \{A[0], \ldots, A[i-1]\}$. If $A[i] \neq x$, the algorithm increments i and we have $x \notin \{A[0], \ldots, A[i]\}$, hence the invariant still holds.

Termination

The loop stops when $A[i] = x$ for some $i \in \{0, \ldots, n-1\}$ or when $i = n$.

- If $A[i] = x$, the algorithm returns its index.
- If $i = n$, the invariant implies x is not in the array.

Therefore the algorithm is correct.